4428 HW4

April 11, 2026

Ram Prasad

[1]:
```
# standard imports
import numpy as np
import matplotlib.pyplot as plt
import warnings

# suppress runtime warnings
warnings.filterwarnings("ignore", category=RuntimeWarning)
```

Q1 (6.5.20)

a.

The continuous time dynamical system

$$
\begin{aligned}
x_1' &= (a-1)x_1 - ax_1^2 \\
x_2' &= x_1 - x_2
\end{aligned}
$$

has equilibrium points where $x_1' = 0$ and $x_2' = 0$ simultaneously. Solving gives

$$
\begin{aligned}
x_2' = 0 &\implies x_1 = x_2 \\
x_1' = 0 &\implies (a-1)x_1 = ax_1^2
\end{aligned}
$$

If $x_1 = 0$, then $(0,0)$ is an equilbrium point. If $x_1 \neq 0$,

$$
\begin{aligned}
(a-1)x_1 &= ax_1^2 \\
a-1 &= ax_1 \\
x_1 &= \frac{a-1}{a}
\end{aligned}
$$

where $a \neq 0$, thus $\boxed{(\frac{a-1}{a}, \frac{a-1}{a})}$ is the other equilibrium point. To assess stability, I compute the Jacobian

$$J = \begin{pmatrix} (a-1) - 2ax_1 & 0 \\ 1 & -1 \end{pmatrix}$$

and evaluate it at the point $(\frac{a-1}{a}, \frac{a-1}{a})$, which gives

$$\begin{aligned} J &= \begin{pmatrix} (a-1) - 2a\frac{a-1}{a} & 0 \\ 1 & -1 \end{pmatrix} \\ &= \begin{pmatrix} (a-1) - 2(a-1) & 0 \\ 1 & -1 \end{pmatrix} \\ &= \begin{pmatrix} 1-a & 0 \\ 1 & -1 \end{pmatrix}. \end{aligned}$$

Using $a > 1$, the eigenvalues are both negative,

$$\begin{aligned} (1-a-\lambda)(-1-\lambda) - 0 = 0 \\ \lambda_1 = 1 - a < 0 \\ \lambda_2 = -1 < 0 \end{aligned}$$

thus the equilibrium point is $\boxed{\text{asymptotically stable}}$.

b.

The discrete time dynamical system

$$\begin{aligned} \Delta x_1 &= x_{1,n+1} - x_{1,n} = (a-1)x_{1,n} - ax_{1,n}^2 \\ \Delta x_2 &= x_{2,n+1} - x_{2,n} = x_{1,n} - x_{2,n} \end{aligned}$$

has as equilibrium points where $\Delta x_1 = 0$ and $\Delta x_2 = 0$, simultaneously. Solving,

$$\begin{aligned} \Delta x_1 = 0 \implies x_{1,n+1} = x_{1,n}, \quad (a-1)x_{1,n} &= ax_{1,n}^2 \\ x_{1,n} &= \frac{a-1}{a} \text{ or } 0. \\ \Delta x_2 = 0 \implies x_{2,n+1} = x_{2,n}, \quad x_{1,n} &= x_{2,n} \end{aligned}$$

Thus $\boxed{\left(\frac{a-1}{a}, \frac{a-1}{a}\right)}$ again is an equilbrium for any $a > 1$. To assess stability, I compute the Jacobian of the mapping

$$\begin{aligned} x_{1,n+1} &= ax_{1,n} - ax_{1,n}^2 \\ x_{2,n+1} &= x_{1,n} \end{aligned}$$

and evaluate it at the equilbrium point, which gives

$$\begin{aligned} J &= \begin{pmatrix} a - 2ax_1 & 0 \\ 1 & 0 \end{pmatrix} \\ &= \begin{pmatrix} a - 2a\frac{(a-1)}{a} & 0 \\ 1 & 0 \end{pmatrix} \\ &= \begin{pmatrix} 2-a & 0 \\ 1 & 0 \end{pmatrix}. \end{aligned}$$

The eigenvalues are $2 - a$ and 0. Asymptotic stability requires

$$|2 - a| < 1 \implies 1 < a < 3.$$

Unlike the continuous time system, which is stable for every $a > 1$, the discrete system is only stable when $\boxed{1 < a < 3}$.

c.

I use Backward Euler simulation to explore the stability of the equilibrium, varying $a = 1.5, 2.0, 2.5, 3.0, 3.5$, and 4.0, and varying the initial condition.

For step size $h > 0$ and iteration k,

$$\begin{aligned} x_1^{k+1} &= x_1^k + h\left((a-1)x_1^{k+1} - a(x_1^{k+1})^2\right) \\ x_2^{k+1} &= x_2^k + h\left(x_1^{k+1} - x_2^{k+1}\right). \end{aligned}$$

Solving

$$\begin{aligned} 0 &= x_1^k + h\left((a-1)x_1^{k+1} - a(x_1^{k+1})^2\right) - x_1^{k+1} \\ 0 &= x_2^k + h\left(x_1^{k+1} - x_2^{k+1}\right) - x_2^{k+1}. \end{aligned}$$

for x_1^{k+1} and x_2^{k+1} gives

$$
\begin{aligned}
x_1^k + [-1 + h(a-1)]x_1^{k+1} - ha(x_1^{k+1})^2 &= 0 \\
-ha(x_1^{k+1})^2 + [-1 + h(a-1)]x_1^{k+1} + x_1^k &= 0 \\
ha(x_1^{k+1})^2 + [1 - h(a-1)]x_1^{k+1} - x_1^k &= 0
\end{aligned}
$$

which is quadratic in x_1^{k+1} with roots

$$
x_1^{k+1} = \frac{-[1 - h(a-1)] \pm \sqrt{[1 - h(a-1)]^2 + 4hax_1^k}}{2ha}.
$$

Similarly,

$$
\begin{aligned}
x_2^k + h(x_1^{k+1}) - (h+1)x_2^{k+1} &= 0 \\
x_2^k + hx_1^{k+1} &= (h+1)x_2^{k+1} \\
x_2^{k+1} &= \frac{x_2^k + hx_1^{k+1}}{h+1}
\end{aligned}
$$

I use the positive root and implement Backward Euler programmatically below.

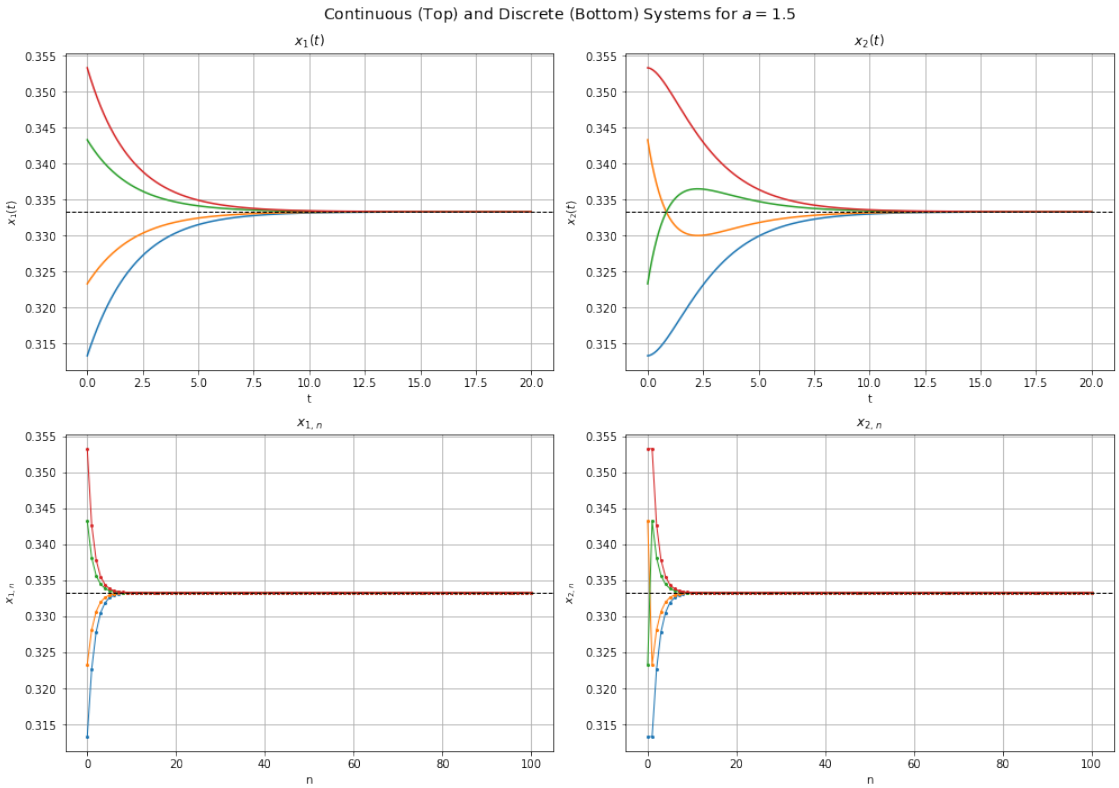
[2]:
```
# define equilibrium
def x_star(a): return (a - 1)/a

# define backward euler step for x1
def x1_BE(x1_k, a, h):
    """
    Backward Euler update for x1. Solves the quadratic
        ha y^2 + [1 - h(a-1)] y - x1_k = 0
    and returns the positive root.
    Args:
    -----
        x1_k (float): current x1 value
        a (float): parameter
        h (float): time step
    Returns:
    --------
        (float): x1_{k+1}
    """
    # quadratic coefficients
    A = h*a
    B = 1 - h*(a - 1)
    C = -x1_k

    # discriminant
    disc = B**2 - 4*A*C
```

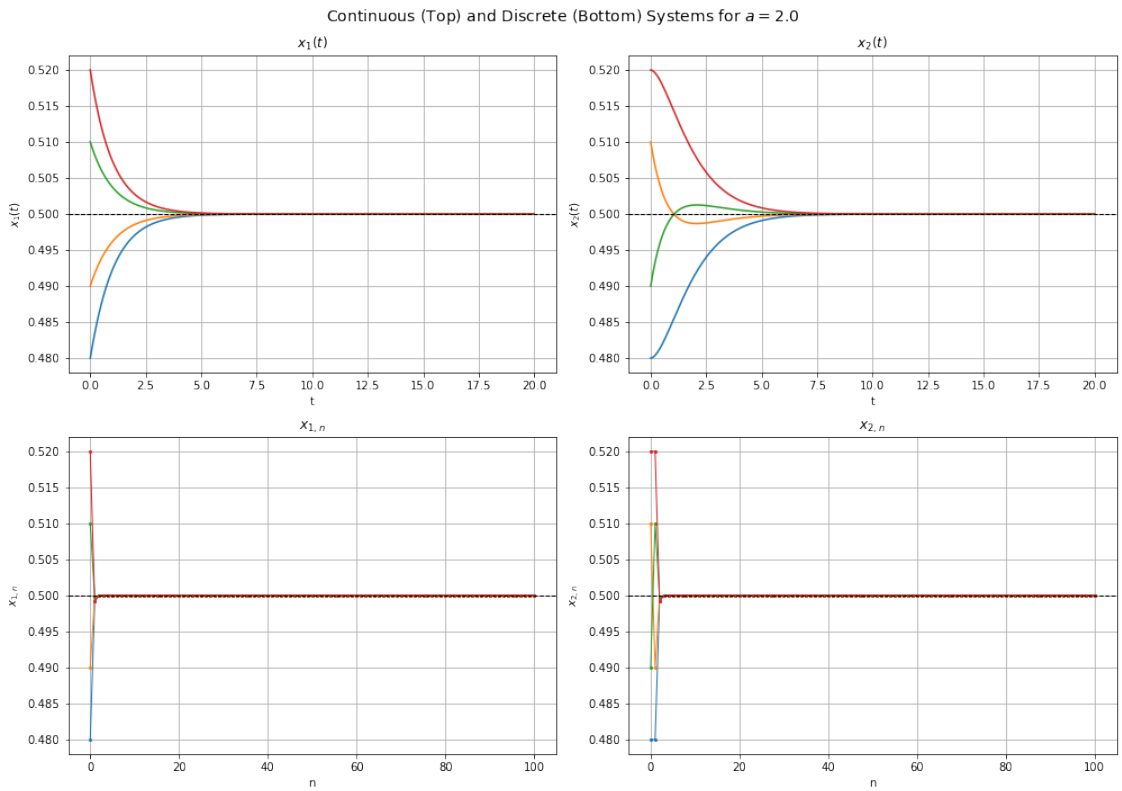
```python
    # positive root
    return (-B + np.sqrt(disc))/(2*A)


# backward euler simulation
def simulate_continuous(x0, a, h=0.05, T=20):
    """
    Simulate the continuous time system using Backward Euler.
    Args:
    -----
        x0 (tuple): initial condition (x1_0, x2_0)
        a (float): parameter
        h (float): step size
        T (float): final time
    Returns:
    --------
        (tuple): time grid, x1 values, x2 values
    """
    # set time grid
    n_steps = int(T/h)
    t_grid = np.linspace(0, T, n_steps + 1)

    # initialize solution arrays
    x1 = np.zeros(n_steps + 1)
    x2 = np.zeros(n_steps + 1)
    x1[0], x2[0] = x0

    # iterate backward euler
    for k in range(n_steps):

        # update x1, x2
        x1[k+1] = x1_BE(x1[k], a, h)
        x2[k+1] = (x2[k] + h*x1[k+1])/(1 + h)

    # return tuple
    return t_grid, x1, x2


# discrete simulation
def simulate_discrete(x0, a, N=100):
    """
    Simulate the discrete-time system
        x1_{n+1} = a x1_n (1 - x1_n)
        x2_{n+1} = x1_n
    Args:
    -----
```

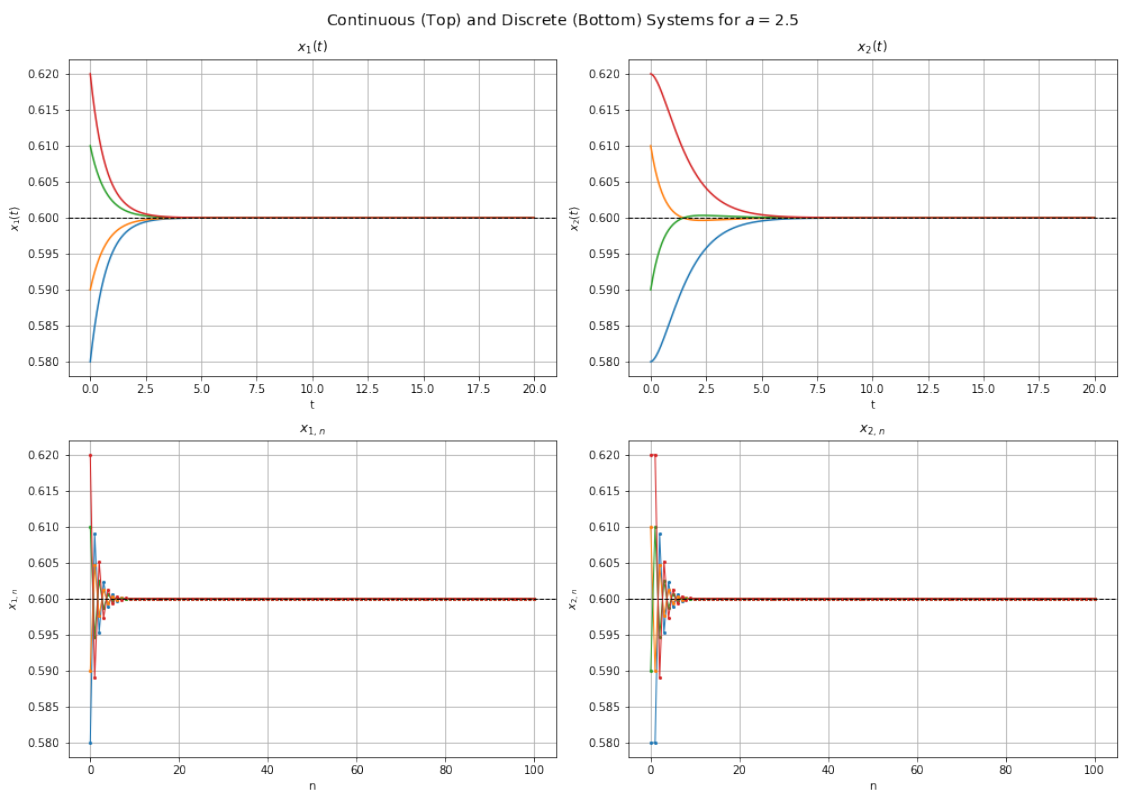
```
        x0 (tuple): initial condition (x1_0, x2_0)
        a (float): parameter
        N (int): number of iterates
    Returns:
    --------
        (tuple): n grid, x1 values, x2 values
    """
    # set iterate grid
    n_grid = np.arange(N + 1)

    # initialize solution arrays
    x1 = np.zeros(N + 1)
    x2 = np.zeros(N + 1)
    x1[0], x2[0] = x0

    # iterate map
    for n in range(N):

        # x1, x2
        x1[n+1] = a*x1[n]*(1 - x1[n])
        x2[n+1] = x1[n]

    # return tuple
    return n_grid, x1, x2

# parameters to test
a_grid = [1.5, 2.0, 2.5, 3.0, 3.5, 4.0]

# nearby initial conditions
pert_grid = [(-0.02, -0.02), (-0.01, 0.01), (0.01, -0.01), (0.02, 0.02)]


# loop through parameter values
for a in a_grid:

    # compute equilibrium
    xs = x_star(a)

    # build initial conditions near eq point
    x0_grid = [(xs + dx1, xs + dx2) for dx1, dx2 in pert_grid]

    # set figure
    plt.figure(figsize=(14, 10))

    # plot continuous x1(t)
    plt.subplot(2, 2, 1)
    for x0 in x0_grid:
```

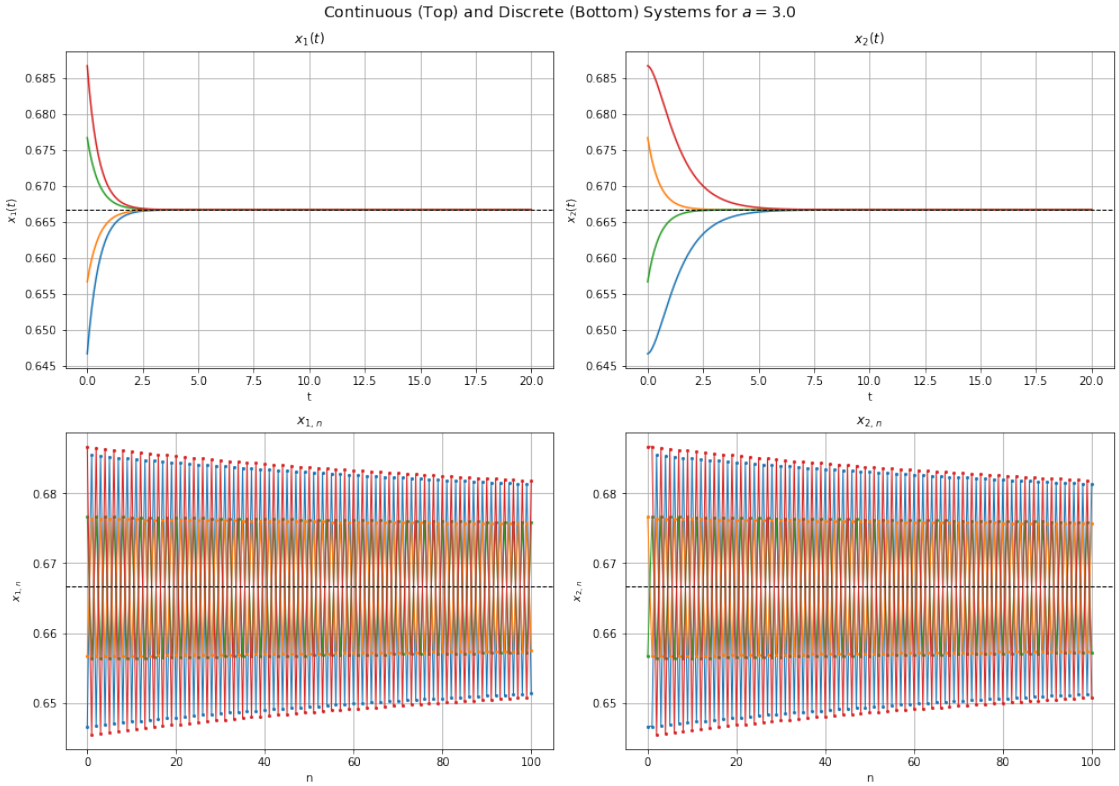
```
        t, x1, x2 = simulate_continuous(x0, a, h=0.05, T=20)
        plt.plot(t, x1, label=fr"$x_0={x0}$")
    plt.axhline(xs, color="black", linestyle="--", linewidth=1)
    plt.xlabel("t")
    plt.ylabel(r"$x_1(t)$")
    plt.title(fr"$x_1(t)$")
    plt.grid()

    # plot continuous x2(t)
    plt.subplot(2, 2, 2)
    for x0 in x0_grid:
        t, x1, x2 = simulate_continuous(x0, a, h=0.05, T=20)
        plt.plot(t, x2, label=fr"$x_0={x0}$")
    plt.axhline(xs, color="black", linestyle="--", linewidth=1)
    plt.xlabel("t")
    plt.ylabel(r"$x_2(t)$")
    plt.title(fr"$x_2(t)$")
    plt.grid()

    # plot discrete x1_n
    plt.subplot(2, 2, 3)
    for x0 in x0_grid:
        n, x1, x2 = simulate_discrete(x0, a, N=100)
        plt.plot(n, x1, marker="o", markersize=2, linewidth=1,␣
↪label=fr"$x_0={x0}$")
    plt.axhline(xs, color="black", linestyle="--", linewidth=1)
    plt.xlabel("n")
    plt.ylabel(r"$x_{1,n}$")
    plt.title(fr"$x_{1,n}$")
    plt.grid()

    # plot discrete x2_n
    plt.subplot(2, 2, 4)
    for x0 in x0_grid:
        n, x1, x2 = simulate_discrete(x0, a, N=100)
        plt.plot(n, x2, marker="o", markersize=2, linewidth=1,␣
↪label=fr"$x_0={x0}$")
    plt.axhline(xs, color="black", linestyle="--", linewidth=1)
    plt.xlabel("n")
    plt.ylabel(r"$x_{2,n}$")
    plt.title(fr"$x_{2,n}$")
    plt.grid()

    # title, display
    plt.suptitle(fr"Continuous (Top) and Discrete (Bottom) Systems for␣
↪$a={a}$", fontsize=14)
    plt.tight_layout()
```

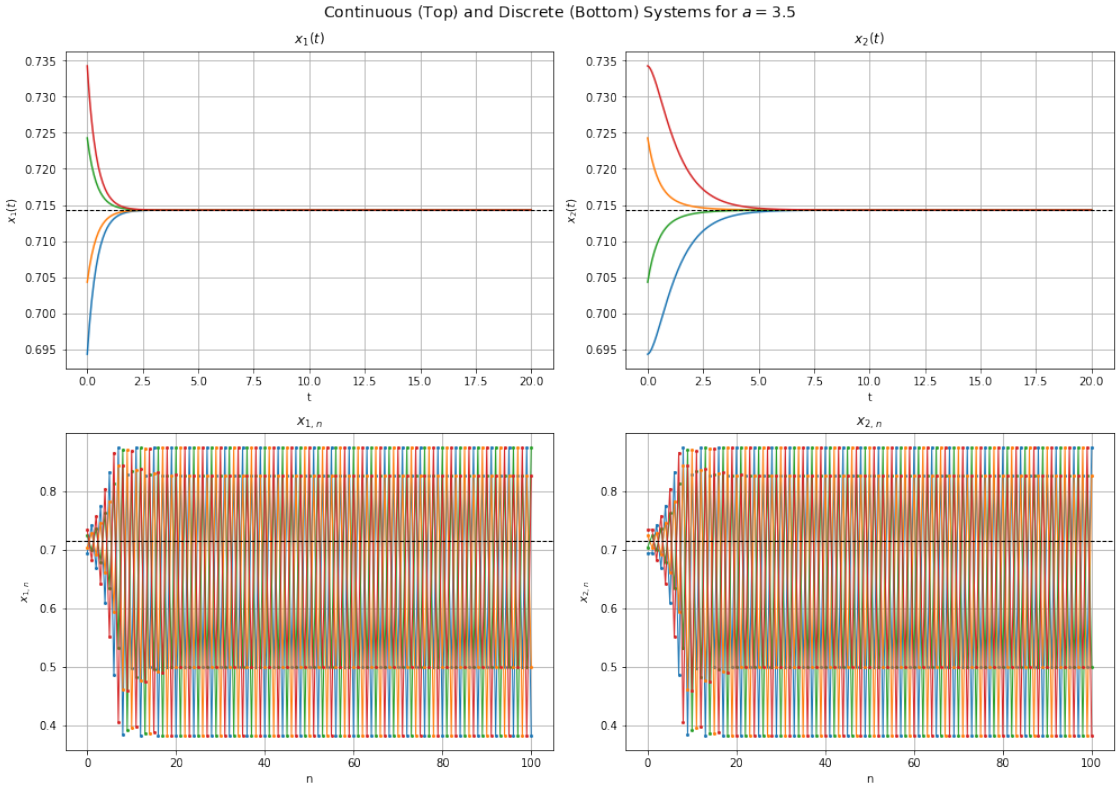
```
plt.show()
```

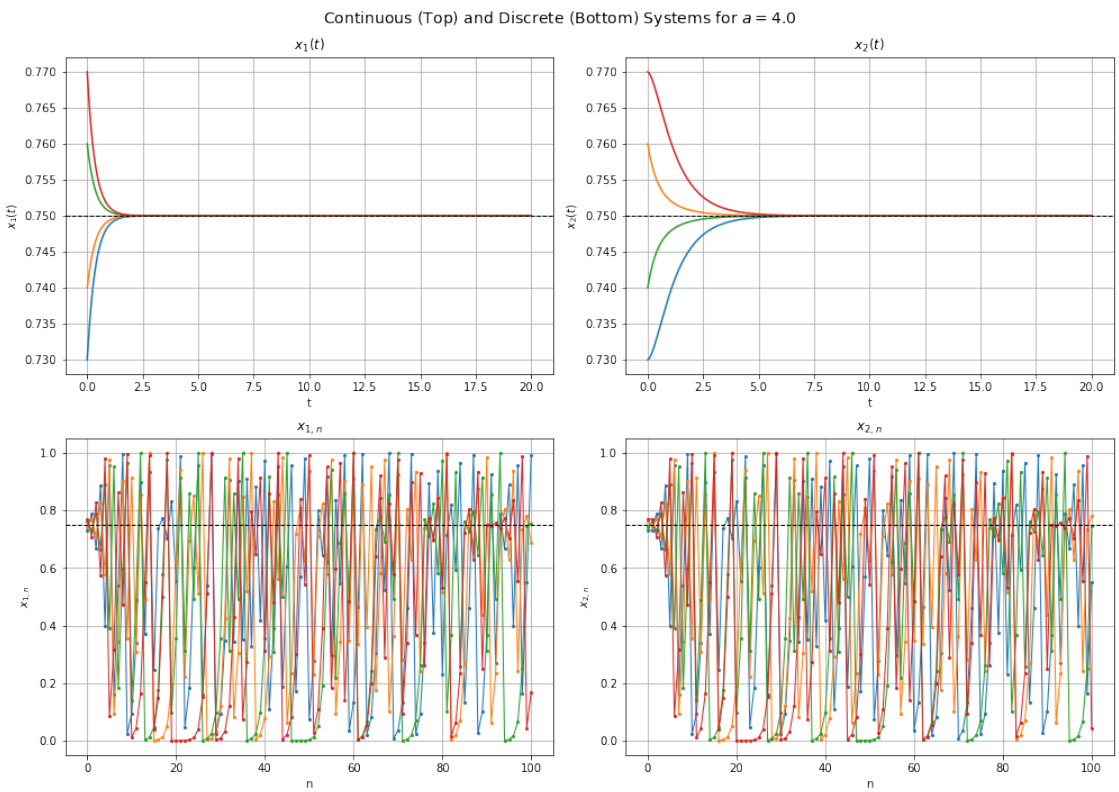

Continuous (Top) and Discrete (Bottom) Systems for $a = 1.5$
$x_1(t)$
$x_2(t)$
$x_{1,n}$
$x_{2,n}$
t
n


Continuous (Top) and Discrete (Bottom) Systems for $a = 2.0$
$x_1(t)$
$x_2(t)$
$x_{1,n}$
$x_{2,n}$
t
n


Continuous (Top) and Discrete (Bottom) Systems for $a = 2.5$
$x_1(t)$
$x_2(t)$
$x_{1,n}$
$x_{2,n}$
t
n


Continuous (Top) and Discrete (Bottom) Systems for $a = 3.0$
$x_1(t)$
$x_2(t)$
$x_1(t)$
$x_2(t)$
t
t
$x_{1,n}$
$x_{2,n}$
$x_{1,n}$
$x_{2,n}$
n
n


Continuous (Top) and Discrete (Bottom) Systems for $a = 3.5$
$x_1(t)$
$x_2(t)$
$x_1(t)$
$x_2(t)$
t
t
$x_{1,n}$
$x_{2,n}$
$x_{1,n}$
$x_{2,n}$
n
n


Continuous (Top) and Discrete (Bottom) Systems for $a = 4.0$
$x_1(t)$
$x_2(t)$
$x_1(t)$
$x_2(t)$
t
t
$x_{1,n}$
$x_{2,n}$
$x_{1,n}$
$x_{2,n}$
n
n

The plots above show $(t, x(t)$ for the continuous time model compared with (n, x_n) for the discrete model. Broadly, the continuous time systems converge to equilibrium for every $a > 1$, but the discrete system's behavior varies with a, progressing from a stable fixed point, to oscillation, periodicity, and then complete chaos. The dashed horizontal line is the equilbrium $(a-1)/a$. Owing to the $x_{2,n+1} = x_{1,n}$ relation in the discrete system, the bottom right panel, which shows discrete $x_{2,n}$ looks like a lagged copy of the bottom left panel, which shows discrete $x_{1,n}$.

I note the following specifically.

First, for $a = 1.5$, the top left graph shows $x_1(t)$ in which all trajectories smoothly approach the equilibrium. There is no oscillation, consistent with a stable equilbrium. Similarly for $x_2(t)$. The discrete iterates quickly approach 1/3 and is stable, consistent with $|2 - 1.5| = 0.5 < 1$. Both systems are stable.

Second, for $a = 2.0$, all continuous $x_1(t)$ trajectories converge to 0.5, and $x_2(t)$ converges as well. The discrete $x_{1,n}$ iterates quickly collapse to the equilbrium.

Third, for $a = 2.5$, the equilbrium is 1.5/2.5=0.6. Continuous $x_1(t)$ and $x_2(t)$ shows smooth convergence. The discrete iterates $x_{1,n}$ oscillate and overshooting before ultimately decaying and settling.

Fourth, for $a = 3.0$, the equilibrium is 2/3. Continuous $x_1(t)$ and $x_2(t)$ converge. Discrete $x_{1,n}$ oscillates and decays very slowly, which is to be expected because $|2 - a| = 1$ isn't quite stable.

Fifth, for $a = 3.5$, the continuous system converges but the discrete iterates don't settle at the dashed equilibrium line. They oscillate at distinct levels, which is consistent with an unstable equilbrium as $|2-a| = 1.5 > 1$. The continuous system remains stable, whereas the discrete system exhibits periodic behavior, exactly as expected.

Sixth, for $a = 4.0$, the continuous $x_1(t)$ and $x_2(t)$ converge to 0.75, whereas $x_{1,n}$ looks irregular and highly sensitive to the initial condition. This is the chaos case for the discrete system which doesn't converge or exhibit simple periodicity.

Q2 (7.5.7)

I assume the following. First, there are 10 possible digits (0-9 inclusive), thus 10^3 possible combinations and there is only one winning number, thus the probability of a ticket winning is $1/10^3$. Second, tickets can be purchased daily. Third, an individual buying multiple tickets gets distinct numbers.

a.

For a single ticket, $p(\text{win}) = \frac{1}{10^3}$. Let W be the number of wins in 52 weeks, and assume that weekly lottery results are independent. Treating a win as a successful trial, W can be modelled as

$$W \sim \text{Bin}\left(52, \frac{1}{10^3}\right).$$

The cost of a ticket is 1 dollar, thus you spend $1 \cdot 52 = 52$ over 52 weeks. I assume that coming out a winner for the year means making more off the lottery than expenditure. For a lottery that

pays winners 500 dollars, the net proceeds are $500W - 52$. Note that because W is integer valued,

$$\begin{aligned} 500W - 52 &> 0 \\ W &> \frac{52}{500} \end{aligned}$$

is equivalent to

$$W \geq 1.$$

Thus the probability of coming out a winner for a year is

$$\begin{aligned} P(W \geq 1) &= 1 - P(W = 0) \\ &= 1 - \binom{52}{0} \left(\frac{1}{10^3}\right)^0 \left(1 - \frac{1}{10^3}\right)^{52} \\ &= \boxed{1 - \left(1 - \frac{1}{10^3}\right)^{52}}. \end{aligned}$$

which evaluates to

[3]:
```
print(1 - (1-1/(10**3))**52)
```

```
0.05069583185373461
```

b.

Now assume that you buy n tickets a week, there is only one winning number per week, and that ticket numbers are distinct. For a single week, the probability of a win is now

$$p(\text{win}) = \frac{n}{10^3}, \qquad n = 1, \dots, 9.$$

Let W_n be the number of wins in 52 weeks, assuming you buy n tickets. Thus

$$W_n \sim \text{Bin}\left(52, \frac{n}{10^3}\right).$$

Total cost is now $52n$, and coming out ahead for the year means

$$\begin{aligned} 500W_n - 52n &> 0 \\ W_n &> \frac{52n}{500} \end{aligned}$$

for $n = 1, \dots, 9$, $\frac{52n}{500} < 1$ (e.g. $n = 9$, $\frac{468}{500} < 1$), thus $W_n > \frac{52n}{500}$ is equivalent to

$$W_n \geq 1.$$

The probability of coming out a winner assuming you buy n tickets a week is thus

$$
\begin{aligned}
P(W_n \geq 1) &= 1 - P(W_n = 0) \\
&= 1 - \left(1 - \frac{n}{10^3}\right)^{52}, \qquad n = 1, \dots, 9.
\end{aligned}
$$

I compute probabilities below.

[4]:
```
for n in range(1,9+1):
    print(f"P(W_{n} >= 1) = {1 - (1 - n/(10**3))**52}")
```

```
P(W_1 >= 1) = 0.05069583185373461
P(W_2 >= 1) = 0.0988685502807084
P(W_3 >= 1) = 0.14464138826863993
P(W_4 >= 1) = 0.18813167449009827
P(W_5 >= 1) = 0.22945111068811785
P(W_6 >= 1) = 0.26870603628913203
P(W_7 >= 1) = 0.3059976808194178
P(W_8 >= 1) = 0.34142240467579144
P(W_9 >= 1) = 0.3750719287769433
```

$\boxed{\text{Yes}}$, you do improve your chances by buying more than one ticket per week.

c.

Assume the lottery sells 1,000,000 tickets, where each ticket independently picks one of the 1000 3 digit numbers. Let W be the number of winning tickets,

$$W \sim \text{Bin}\left(10^6, \frac{1}{10^3}\right).$$

Let the lottery's profit be X, which is the difference between revenue (1MM tickets at a dollar each) and cost (500 dollars paid out for each winner), thus

$$X = 1{,}000{,}000 - 500W.$$

To apply the CLT to X, I first compute moments.

$$
\begin{aligned}
E(W) &= 10^6 \cdot \frac{1}{10^3} \\
&= 1000 \\
\mathrm{Var}(W) &= 10^6 \cdot \frac{1}{10^3}\left(1 - \frac{1}{10^3}\right) \\
&= 1000(0.999) \\
&= 999 \\
E(X) &= 1{,}000{,}000 - 500E(W) \\
&= 1{,}000{,}000 - 500(1000) \\
&= 500{,}000 \\
\mathrm{Var}(X) &= 500^2\,\mathrm{Var}(W) \\
&= 500^2(999) \\
\sigma_X &= 500\sqrt{999} \approx 15{,}803.
\end{aligned}
$$

A binomial random variable is the sum of independent Bernoulli trials, e.g.

$$
W = W_1 + \cdots + W_{10^6}, \quad W_i \sim \text{Bernoulli}(\frac{1}{10^3}),
$$

thus by the CLT, W can be approximated using a normal distribution with mean 1000 and standard deviation $\sqrt{999}$, and

$$
X \approx N\big(500{,}000,\ 500^2(999)\big)\,.
$$

To compute a 95% confidence interval for the state's weekly profit, I use the construction of mean plus or minus critical value times standard deviation. Thus the range of likely variation is

$$
\begin{aligned}
500{,}000 \pm 2(15{,}803) &= 500{,}000 \pm 31{,}606 \\
&\approx \boxed{(468{,}394,\ 531{,}606)}.
\end{aligned}
$$

The state loses money when $X < 0$, which occurs when

$$
\begin{aligned}
X &< 0 \\
1{,}000{,}000 - 500W &< 0 \\
W &> 2000,
\end{aligned}
$$

meaning there are more than 2000 winners. Standardizing,

$$
\begin{aligned}
P(X<0) &= P\left(\frac{X-500{,}000}{500\sqrt{999}} < \frac{0-500{,}000}{500\sqrt{999}}\right) \\
&\approx P(Z<-31.6).
\end{aligned}
$$

Thus the state will not lose money this week because the probability of more than 2000 winners is effectively 0.

d.

Using the CLT in part (a) or (b) is wrong because in the number of trials is only 52, and the success probability is very small, so neither of the rules of thumb for the normal approximation to the binomial (e.g. $np \geq 10, n(1-p) \geq 10$) is satisfied. In (c), the CLT is applicable since both criteria are easily satisfied.

Q3 (7.5.8)

Taxis arrive randomly, at a rate of 1 per 5 minutes, thus I assume that taxi cab interarrival times $X \sim \text{Exp}(\lambda)$ where $\lambda = 0.2$. The number of arrivals in the interval $[0, t]$ is thus a Poisson process with $\lambda = 0.2$.

a.

By the memoryless property, the time to wait is a fresh exponential

$$\boxed{E(X) = \frac{1}{\lambda} = 5}.$$

b.

The forward recurrence time is distributed $\text{Exp}(\lambda)$, thus the backward recurrence time is distributed $\text{Exp}(\lambda)$, thus the expected time since the most recent arrival is

$$\boxed{E(\text{backward}) = \frac{1}{\lambda} = 5}.$$

c.

Let B be the backward recurrence time and F the forward recurrence time. The interval between arrival of the just missed cab and the next cab is the sum of recurrence times, $B + F$.

$$
\begin{aligned}
E(B+F) &= E(B) + E(F) \\
&= \frac{1}{\lambda} + \frac{1}{\lambda} \\
&= 5 + 5 = \boxed{10},
\end{aligned}
$$

which is not the expected interarrival time (5). This result is called the inspection paradox. The interarrival time has mean 5, but the interarrival interval containing a randomly chosen time has mean 10 on account of length bias (if you pick a random interval, it is more likely to be a longer interval than a shorter one).

Q4 (7.5.14)

Let $X \sim \text{Geom}(p)$ with PMF

$$\Pr(X = i) = p(1-p)^{i-1}, \qquad i = 1, 2, ...$$

a.

Using the PMF,

$$\Pr(X = i) = p(1-p)^{i-1}, \quad i = 1, 2, 3, ...$$

I compute

$$\begin{aligned}
\Pr(X > i) &= \Pr(X = i+1) + \Pr(X = i+2) + \cdots \\
&= \sum_{j=1}^{\infty} \Pr(X = i+j) \\
&= \sum_{j=1}^{\infty} p(1-p)^{i+j-1} \\
&= \sum_{j=1}^{\infty} p(1-p)^{i}(1-p)^{j-1} \\
&= p(1-p)^{i} \sum_{j=1}^{\infty} (1-p)^{j-1} \\
&= p(1-p)^{i} \left(1 + (1-p) + (1-p)^2 + \cdots\right) \\
&= p(1-p)^{i} \cdot \frac{1}{1-(1-p)} \\
&= p(1-p)^{i} \cdot \frac{1}{p} \\
&= \boxed{(1-p)^{i}}
\end{aligned}$$

since 0<p<1, the geometric series converges.

b.

I demonstrate the memoryless property using the result from (a),

$$
\begin{aligned}
\Pr(X > i + j \mid X > j) &= \frac{\Pr(X > i + j \cap X > j)}{\Pr(X > j)} \\
&= \frac{\Pr(X > i + j)}{\Pr(X > j)} \\
&= \frac{(1-p)^{i+j}}{(1-p)^{j}} \\
&= (1-p)^{i} \\
&= \boxed{\Pr(X > i)}.
\end{aligned}
$$

c.

Note that taking the derivative of the geometric sum $1 + x + x^2 + x^3 + \cdots = \frac{1}{1-x}$ gives

$$1 + 2x + 3x^2 + \cdots = (1-x)^{-2}.$$

I compute $E(X)$.

$$
\begin{aligned}
E(X) &= \sum_{i=1}^{\infty} i \Pr(X = i) \\
&= \sum_{i=1}^{\infty} i \cdot p(1-p)^{i-1} \\
&= p \sum_{i=1}^{\infty} i(1-p)^{i-1} \\
&= p\left(1 + 2(1-p) + 3(1-p)^2 + 4(1-p)^3 + \cdots\right) \\
&= p(1-(1-p))^{-2} \\
&= p \cdot p^{-2} = \boxed{\frac{1}{p}}
\end{aligned}
$$

In discrete time, the arrival rate is the mean number of arrivals per time step, thus because the expected interarrival time is $E(X) = 1/p$, the interrarrival time X has rate $\frac{1}{E(X)} = p$.

d.

Using a rate of one every 10 minutes ($\lambda = \frac{1}{10}$) and 1 minus the CDF of an exponential random variable,

$$
\begin{aligned}
\Pr(Y > 5) &= 1 - \Pr(Y \leq 5) \\
&= 1 - (1 - e^{-\lambda 5}) \\
&= e^{-(1/10)5} \\
&= \boxed{e^{-1/2}}.
\end{aligned}
$$

Using a geometric model with $p = \frac{1}{10}$,

$$\Pr(X > 5) = (1-p)^5 = \boxed{\left(\frac{9}{10}\right)^5}$$

[5]:
```
# y>5
print(f"P(Y>5): {np.exp(-0.5)}")

# x>5
print(f"P(X>5): {0.9**5}")
```

```
P(Y>5): 0.6065306597126334
P(X>5): 0.5904900000000001
```

The probability that $Y > 5$ is marginally higher.